
Mastering Data Structures and Algorithms in Python – A 6-Week Roadmap

 **Author: Samwel Ngusa**

 **Language: Python 3**

 **Duration: 6 Weeks**

 **Learning Style: Book-First + Practice-Heavy**

Project Type: Public GitHub Study Log

Mastering Data Structures and Algorithms in Python – A 6-Week Roadmap..... 1

 Author: Samwel Ngusa.....	1
 Language: Python 3.....	1
 Duration: 6 Weeks.....	1
 Learning Style: Book-First + Practice-Heavy.....	1
 Project Type: Public GitHub Study Log.....	2
 Why Learn DSA?.....	4
 Benefits:.....	4
 Tools & Resources.....	4
 The 6-Week DSA Master Plan.....	5
 Week 1: DSA Foundation & Python Prerequisites.....	5
 Goals:.....	5
 Topics:.....	5
 Action Plan:.....	5
 Week 2: Basic Data Structures.....	6
 Goals:.....	6
 Topics:.....	6
 Action Plan:.....	6
 Tips:.....	6
 Week 3: Hashing, Recursion & Backtracking.....	7
 Goals:.....	7
 Topics:.....	7
 Action Plan:.....	7
 Week 4: Advanced Data Structures.....	7
 Goals:.....	8
 Topics:.....	8
 Action Plan:.....	8
 Practice:.....	8
 Week 5: Mastering Problem-Solving Patterns.....	9
 Goals:.....	9
 Patterns:.....	9
 Action Plan:.....	9
 Tips:.....	9
 Week 6: Projects + Interview Prep.....	10
 Goals:.....	10
 Mini Projects:.....	10
 Interview Focus:.....	10
 Core Problem-Solving Patterns Summary.....	10

	How to Track Progress.....	11
	Final Thoughts.....	12
	License & Contribution.....	12

Why Learn DSA?

Before diving into Data Structures and Algorithms (DSA), it's crucial to understand **why** you're doing it. DSA isn't just an academic requirement — it's a foundational skill for any software engineer or developer.

✨ Benefits:

-  **Interviews:** Companies like Google, Amazon, Meta, and startups focus heavily on DSA during technical interviews.
-  **Problem Solving:** DSA improves your logic, reasoning, and critical thinking.
-  **Performance:** A strong grasp of DSA helps you build faster, more efficient programs.
-  **Real-World Coding:** Everything from search engines to social media algorithms relies on efficient data structures.

Tools & Resources

Category	Tool
 Main Book	<i>Hands-On Data Structures and Algorithms with Python</i> by Basant Agarwal
 Video Support	NeetCode, Abdul Bari (YouTube)
 Practice Sites	LeetCode, HackerRank, GeeksforGeeks

 Tracking Notion, Google Sheets, Notebook

 IDEs VS Code, Jupyter Notebook, PyCharm

The 6-Week DSA Master Plan

Each week builds on the previous one, covering both **theory and practical coding exercises**. Stick to it daily, spend **2.5–3 hours a day**, and review tough problems regularly.

Week 1: DSA Foundation & Python Prerequisites

Goals:

- Understand why DSA is essential.
- Review Python fundamentals.
- Learn Big-O and complexity basics.

Topics:

- Lists, tuples, dictionaries, sets
- Functions, loops, recursion
- Time and space complexity (Big-O notation)

Action Plan:

- Read **Chapters 1–2** from the book.

- Solve 5 Python warm-up problems on HackerRank.
 - Watch NeetCode's "Big-O Notation Explained".
-

Week 2: Basic Data Structures

Goals:

- Learn and implement foundational data structures.

Topics:

- Arrays & Strings
- Stacks & Queues
- Linked Lists (Singly, Doubly, Circular)

Action Plan:

- Book: **Chapters 3–5**
- Implement each structure from scratch.
- Solve problems:
 - Two Sum
 - Valid Parentheses
 - Reverse Linked List
 - Merge Two Lists

Tips:

- Focus on **how** data is added, removed, and accessed.

- Try solving each problem **without peeking** at solutions first.
-

Week 3: Hashing, Recursion & Backtracking

Goals:

- Learn hashing for fast lookup.
- Master recursion and its stack-based behavior.
- Begin solving backtracking problems.

Topics:

- Hash maps and sets
- Recursive functions and patterns
- Backtracking (exhaustive search)

Action Plan:

- Book: **Chapter 6**
 - Solve:
 - Group Anagrams
 - Longest Substring Without Repeating
 - Word Search
 - Sudoku Solver
-

Week 4: Advanced Data Structures

Goals:

- Work with more complex structures used in real systems.

Topics:

- Binary Trees, Binary Search Trees (BST)
- Heaps (Min/Max)
- Graphs (BFS, DFS)
- Tries (Prefix Trees)

Action Plan:

- Book: **Chapters 7–10**
- Implement:
 - Tree Traversals: Inorder, Preorder, Postorder
 - MinHeap/MaxHeap from scratch
 - Graph using Adjacency List and Matrix
 - Trie insert and search functions

Practice:

- Diameter of Binary Tree
 - Kth Largest Element in Stream
 - Clone Graph
 - Longest Word in Dictionary (Trie)
-

Week 5: Mastering Problem-Solving Patterns

Goals:

- Recognize common coding patterns to speed up problem solving.

Patterns:

- Two Pointers
- Sliding Window
- Binary Search
- Greedy
- Backtracking
- Dynamic Programming (Intro)

Action Plan:

- Study 1–2 patterns per day with examples.
- Solve key problems:
 - Container With Most Water (Two Pointers)
 - Longest Substring Without Repeat (Sliding Window)
 - Search in Rotated Sorted Array (Binary Search)
 - Jump Game (Greedy)
 - Climbing Stairs (DP)

Tips:

- Build your own **cheat sheet** for each pattern.

- Always review **why** the pattern works.
-

Week 6: Projects + Interview Prep

Goals:

- Apply DSA knowledge in real-world scenarios.
- Start mock interview practice.

Mini Projects:

-  **Spell Checker** → Trie + Hashing
-  **Pathfinding Visualizer** → Graph (BFS/DFS)
-  **Event Scheduler** → MinHeap for time tracking

Interview Focus:

- Start solving **Blind 75** on LeetCode
 - Re-solve difficult problems from past weeks
 - Join mock interviews (e.g. Pramp, Interviewing.io)
-

Core Problem-Solving Patterns Summary

Pattern	Examples
Sliding Window	Longest Substring w/o Repeat

Two Pointers	Container With Most Water
--------------	---------------------------

Binary Search	Search in Rotated Sorted Array
---------------	--------------------------------

Backtracking	N-Queens, Sudoku
--------------	------------------

Greedy	Gas Station, Jump Game
--------	------------------------

Dynamic Programming	Fibonacci, 0/1 Knapsack, LCS
---------------------	------------------------------



How to Track Progress

Method	Tool
--------	------

Problem Log	Google Sheet / Notion
-------------	-----------------------

Revision	Weekly re-solving schedule
----------	----------------------------

Code Notes	VS Code Snippets / Notebook
------------	-----------------------------

Accountability	Join a Discord group or GitHub project
----------------	--

✓ Final Thoughts

- ✓ **Consistency matters more than speed.** Even 2 problems a day will get you there.
 - 📈 **Repetition = Retention** — revisit tricky problems after 1 week and 1 month.
 - 🧱 **Master the patterns**, and you'll solve 80% of interview problems faster.
 - 🧑 **DSA is a long-term investment** — it pays off in interviews, coding contests, and real projects.
-

📢 License & Contribution

This roadmap is designed for public use. Fork it, clone it, remix it — just don't forget to commit consistently! Contributions and improvements are always welcome.

Let me know if you'd like this:

- In **Markdown format** for GitHub?
- Exported as a **PDF** for offline use?
- As a **Notion workspace template** for tracking?